

Assignment - 4

(The code is below the Question)

Use class, objects, constructors while coding

Bank Account Management System

You are tasked with creating a simple bank account management system in Python.

Implement a class called BankAccount with the following specifications:

The class should have private instance variables for account number, account holder name, and balance.

Include a constructor to initialize these variables.

Implement getter and setter methods for each instance variable to ensure encapsulation.

Implement methods to deposit and withdraw money from the account.

Ensure that the withdrawal method checks if the account has sufficient balance before allowing withdrawal.

Write a Python program to demonstrate the functionality of the BankAccount class by creating instances, depositing and withdrawing money, and displaying account information.

Code:

```
# Bank Account Management System

class BankAccount:
    def __init__(self, acc_num, name, initial_balance):
        self.__account_number = acc_num
        self.__acc_holder_name = name
        self.__balance = initial_balance

    # Getter Methods
    def get_name(self):
        return self.__acc_holder_name

    def get_balance(self):
        return self.__balance
```

```

def get_acc_number(self):
    return self.__account_number

# Setter Methods
def set_name(self, new_name):
    self.__acc_holder_name = new_name
    return self.__acc_holder_name

# Deposition Logic
def deposit(self, amount):
    self.__balance = self.__balance + amount
    print('Successfully deposited amount is: ', amount)
    print('The total balance is: ', self.__balance)

# Logic for withdrawing money
def withdraw(self, amount):
    if amount <= self.__balance:
        self.__balance = self.__balance - amount
        print("Successfully withdrew: ", amount)
        print("The remaining balance is: ", self.__balance)
    else:
        print("Sorry!! You don't have enough money.")

# Driver code

my_account = BankAccount("13786254", "Dara Shinunora", 5000)

print("Account Holder Name: ", my_account.get_name())
print("Account Number: ", my_account.get_acc_number())
print("Account Balance: ", my_account.get_balance())

print("The Updated name of Account holder is: ", my_account.set_name("Sara Dune"))

my_account.deposit(500)
my_account.withdraw(200)
my_account.withdraw(7000)

print("The Final Balance: ", my_account.get_balance())

```

Output:

```
Account Holder Name:  Dara Shinunora
Account Number:  13786254
Account Balance:  5000
The Updated name of Account holder is:  Sara Dune
Successfully deposited amount is:  500
The total balance is:  5500
Successfully withdrew:  200
The remaining balance is:  5300
Sorry!! You don't have enough money.
The Final Balance:  5300
```

Employee Management System

You are developing an employee management system for a company. Implement a class called Employee with the following specifications:

The class should have private instance variables for employee ID, name, and salary. Include a constructor to initialize these variables.

Implement getter and setter methods for each instance variable.

Implement a method to display employee information. Implement a method to give a salary hike to an employee.

The method should accept a percentage value by which the salary will be increased.

Write a Python program to demonstrate the functionality of the Employee class by creating instances, displaying employee information, giving salary hikes, and displaying updated employee information.

Code:

```
class Employee:
    def __init__(self, emp_id, name, salary):
        self.__emp_id = emp_id
        self.__name = name
        self.__salary = salary

    # Getter and Setter
    def get_salary(self):
        return self.__salary
```

```

def set_salary(self, new_salary):
    self.__salary = new_salary

# Method to show all info at once
def display_info(self):
    print("Employee ID:", self.__emp_id)
    print("Name:", self.__name)
    print("Salary:", self.__salary)
    print("-----")

# Salary Hike by a percentage
def give_salary_hike(self, percentage):
    increase = self.__salary * (percentage / 100)
    self.__salary = self.__salary + increase
    print("Hike of", percentage, "% applied!")

# Driver Code
emp1 = Employee("E101", "Moana Duke", 50000)

print("Original Info:")
emp1.display_info()

# Give a 10% hike
emp1.give_salary_hike(10)

print("Updated Info:")
emp1.display_info()

```

Output:

```

Original Info:
Employee ID: E101
Name: Moana Duke
Salary: 50000
Hike of 10 % applied!
Updated Info:
Employee ID: E101
Name: Moana Duke
Salary: 55000.0

```